

# Allocation Dynamique de Tâches Multi-Robots

Lionel Alves Vieira, Alexis Dufossé, and Laurent Chen

Sorbonne Université

21 décembre 2025

## Résumé

Cette synthèse propose une analyse des approches récentes et fondatrices du problème d'allocation dynamique de tâches dans les systèmes multi-robots, en se focalisant sur les contraintes et les incertitudes.

## 1 Introduction

Les systèmes multi-robots connaissent aujourd'hui un essor considérable dans de nombreux domaines applicatifs, allant de la logistique automatisée à l'exploration de structures encore inconnues, en passant par la surveillance, la recherche-secours (*search and rescue*) ou encore l'assistance aux humains. Dans ces environnements, l'un des défis scientifiques clés réside dans la capacité à coordonner efficacement un ensemble d'agents robotiques autonomes pour accomplir des tâches potentiellement variées, distribuées entre les robots et dans leur **environnement**, tout en étant **dynamique** et évolutive dans le **temps**. Ce problème, connu sous le nom d'**allocation de tâches multi-robot** (*Multi-Robot Task Allocation*, **MRTA**), constitue un champ central de la robotique, notamment à cause de sa difficulté computationnelle - souvent NP-difficile - et de son importance dans ses applications.

L'objectif général du MRTA est d'assigner à chaque robot un sous-ensemble de tâches de manière à **optimiser** un critère global donné (coût, temps d'exécution et/ou de réalisation, énergie, couverture, robustesse, etc.) tout en respectant des **contraintes** inhérentes au domaine ou explicitées par l'utilisateur. Une première formalisation rigoureuse des différents variantes du problème (notamment d'où vient le terme MRTA) a été proposée par *Gerkey* et *Matarić* [3], dont la nomenclature (ST-SR / MT-MR / etc.) distingue notamment les tâches simples ou multiples, la capacité des agents à exécuter simultanément plusieurs actions, ainsi que les différents types d'allocation (unique, multiple, instantanée, continue, etc.). Cela n'empêche pas cette taxonomie d'évoluer encore aujourd'hui, par exemple avec le terme introduit cette année par *Eskeri*, *Kyrki*, *Baumann* et *Kucner* [8] pour décrire une allocation avertie des contraintes humaines, HATA. Cette grille d'analyse reste donc même aujourd'hui une référence.

Dès la fin des années 1990, les travaux fondateurs de *Parker* et son architecture ALLIANCE [11], ont mis en évidence l'importance d'intégrer la robustesse aux défaillances et les différences entre les agents dans les méca-

nismes d'MRTA. Par la suite, une approche très répandue s'est appuyée sur des principes issus de la science économique : les méthodes d'**allocation par enchères**, synthétisées chez *Dias*, *Stentz et al* [2]. Elles sont extrêmement utilisées dans les papiers récents. Ces méthodes ont marqué un tournant dans ce domaine en introduisant des mécanismes d'optimisation distribués conciliant centralisation et décentralisation (allocation par enchères), efficacité et modularité.

Les évolutions récentes du problème, quant à elles, témoignent d'une gestion plus pointilleuse de la MRTA en ayant des problématiques plus complexes, souvent liées à l'**échelle**, à l'**incertitude**, ou à la **prise en compte de contraintes supplémentaires**. Plusieurs travaux contemporains se sont intéressés à un nombre beaucoup plus important de robots [6], tandis que d'autres intègrent des contraintes et incertitudes temporelles [15], spatiales ou de collision [18] dans le processus d'allocation. De plus, récemment certaines approches intègrent même l'humain dans leur processus de décision [8]. Une revue complète récente du domaine [1] montre ainsi que la tendance actuelle n'est plus simplement de résoudre l'allocation des tâches, mais de répondre à des environnements dynamiques, incertains et sous contraintes.

Dans cette optique, un moyen pertinent pour synthétiser l'état de l'art consiste à structurer l'analyse selon les **types de contraintes** et les **incertitudes** auxquelles les systèmes multi-robots font face. En effet, une large partie des contributions scientifiques cherchent avant tout à répondre à la question de comment allouer des tâches efficacement lorsque le système est soumis à des contraintes temporelles, environnementales ou de mouvement, mais aussi lorsque l'environnement, la dynamique des tâches ou le comportement des autres agents (humains, robots, etc.) sont partiellement connus, probabilistes, évolutifs ou dynamiques. Cette structuration par contraintes et incertitudes permet non seulement de mettre en lumière les défis de chaque famille de problème, mais aussi d'être **critique** quant aux méthodologies employées pour les traiter.

Notre synthèse adoptera donc cette logique : après une présentation détaillée d'un article d'une approche de référence et de son algorithme : SCOPA [15]. Nous analyserons plusieurs approches sélectionnées par rapport à leur type de contraintes et incertitudes. L'objectif est de proposer une synthèse critique de l'état de l'art et des méthodes existantes, afin d'identifier leur convergences et divergences et de dégager les tendances actuelles du domaine. Enfin, une conclusion mettra en évidence les li-

mites, ainsi que les questions ouvertes qui influencent la recherche actuelle sur l'allocation dynamique de tâches dans les systèmes multi-robots.

## 2 Approche de référence : SCoBA [15]

Dans cette synthèse, nous examinons de près l'algorithme **SCoBA** (*Stochastic Conflict-Based Allocation*). Cet algorithme reflète une spécificité importante : au lieu de considérer l'allocation comme une simple correspondance entre robots et tâches à un instant donné, il traite le problème comme une planification détaillée dans le temps, avec des **contraintes de fenêtre temporelle** et une **incertitude sur l'issue des tâches**.

### 2.1 Architecture Hiérarchique

L'innovation principale de SCoBA réside dans son architecture qui découple deux problèmes mathématiquement complexes pour les rendre plus traitables : la **planification** stochastique individuelle (haut-niveau) et la **coordination** multi-agents (bas-niveau).

**Low-Level : Single-Agent Policy Generation :** Le low-level se concentre sur un agent unique, ignorant totalement la présence des autres robots. Son objectif est de générer une politique -quelle action chaque agent doit effectuer- qui **minimise** l'espérance de **pénalité** de l'agent. Pour ce faire, l'algorithme construit un **Arbre de Politique** (*Policy Tree*) en balayant l'axe temporel.

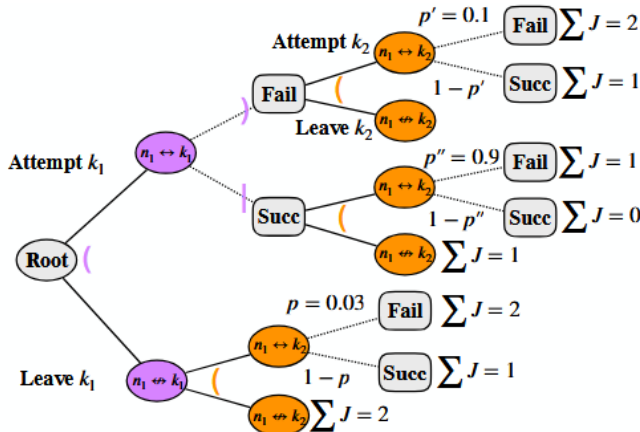


FIGURE 1 – Schéma d'un arbre de politique pour un agent  $n_1$  et deux tâches  $k_1$  et  $k_2$  durant une fenêtre de temps donnée. Source : [15]

Cet arbre contient deux types de nœuds :

- **Nœuds de décision** : Au début d'une fenêtre de temps, l'agent choisit de *Tenter* ( $n_1 \leftrightarrow k_1$ ) ou de *Laisser* ( $n_1 \leftrightarrow k_1$ ) une tâche (ici en violet sur notre Figure 1), puis continue pour ( $n_1 \leftrightarrow k_2$  et  $n_1 \leftrightarrow k_2$ ) en orange.
- **Nœuds de résultat** : Si une tâche est tentée, le résultat est probabiliste (*Succès* ou *Échec*) (ici en gris sur la Figure 1).

L'agent utilise la programmation dynamique pour propager les valeurs des feuilles vers la racine. Pour un nœud de résultat, la valeur est l'espérance pondérée :

$$V(\text{parent}) = p \cdot V(\text{Fail}) + (1 - p) \cdot V(\text{Succ})$$

Pour un nœud de décision, l'agent fait le choix optimal en minimisant l'espérance des fils du nœud :

$$V(\text{parent}) = \min\{V(\text{child}_1), V(\text{child}_2)\}$$

Cette étape garantit que chaque agent dispose de la meilleure stratégie "égoïste" possible face à l'incertitude.

**High-Level : Multi-Agent Coordination** Le high-level utilise une adaptation de la méthode *Conflict-Based Search* (CBS) pour gérer les interactions. Au lieu de planifier pour l'ensemble de la flotte (ce qui exploserait en complexité), ce niveau cherche uniquement à résoudre les **conflits**. Un conflit est défini par l'intersection des politiques de deux agents (par exemple  $n_1$  et  $n_3$ ) sur la même tâche  $k_1$  durant des fenêtres de temps chevauchantes.

SCoBA construit alors un **Arbre de Contraintes** (*Constraint Tree*) :

1. Si un conflit  $\{n_1, n_3\}$  sur  $k_1$  est détecté, le nœud courant est divisé en deux branches (scénarios).
2. **Branche Gauche** : On ajoute la contrainte "*Agent  $n_3$  cannot take  $k_1$* ". L'agent  $n_3$  est forcé de replanifier sans la tâche  $k_1$ .
3. **Branche Droite** : On ajoute la contrainte "*Agent  $n_1$  cannot take  $k_1$* ". L'agent  $n_1$  est forcé de replanifier (via le Low-Level) sans la tâche  $k_1$ .

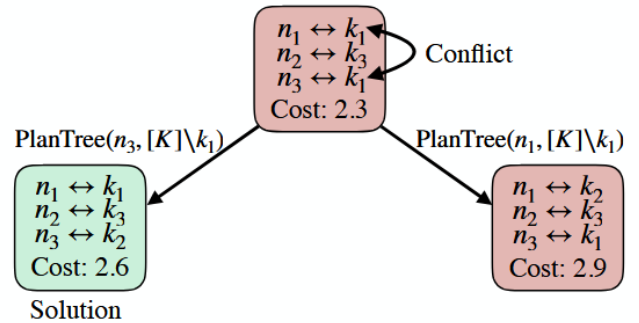


FIGURE 2 – Schéma d'un arbre de contraintes gérant un conflit entre deux agents  $n_1$  et  $n_3$  sur la tâche  $k_1$ . Source : [15]

L'algorithme explore cet arbre avec une stratégie *Best-First Search*, garantissant de trouver la combinaison de contraintes qui minimise le coût global (ici la meilleure solution est représentée en vert sur la Figure 2).

### 2.2 Fonction Objectif et Optimisation

L'objectif global est de **minimiser le nombre espéré de tâches non réalisées** (ou en retard). Pour formaliser cela, définissons :

- $\Pi$  : l'ensemble de toutes les politiques d'allocation conjointes possibles pour la flotte de robots.

- $\pi \in \Pi$  : une politique spécifique qui assigne des séquences de tâches aux agents.

La fonction objectif  $J(\pi)$  représente le coût total espéré de cette politique  $\pi$ , défini comme la somme des pénalités des tâches échouées.

La force de SCoBA est que la fonction de coût global  $J(\pi)$  est simplement la somme des coûts optimaux individuels calculés par le bas-niveau :

$$\min_{\pi \in \Pi} J(\pi) = \sum_n V(\text{root}_n)$$

où  $V(\text{root}_n)$  est la valeur à la racine de l'arbre de politique de l'agent  $n$ .

Cette propriété additive permet une communication efficace : les agents n'ont besoin de transmettre que leur coût espéré et leurs conflits potentiels au coordinateur, préservant ainsi la modularité.

## 2.3 Résultats Expérimentaux et Performance

La performance de SCoBA a été évaluée en simulation sur deux scénarios réalistes : un système de bras robotisés sur un tapis roulant (*Conveyor Belt*) et une flotte de drones de livraison. Les auteurs comparent leur approche à plusieurs références : des heuristiques gloutons (*EDD*, [12] *Algorithme Hongrois* [20]) et des méthodes de planification (*MCTS*, *Q-Learning* [5]).

Les résultats, illustrés par les trois graphiques ci-dessous (issus du papier), montrent la fraction de tâches manquées (objets ratés ou livraisons en retard) en fonction de trois paramètres clés (voir Fig. 3) :

1. **Probabilité de Succès (Grasp Probability)** : Même lorsque la fiabilité des robots diminue (incertitude élevée), SCoBA maintient un taux d'échec nettement inférieur aux autres méthodes, prouvant la robustesse de sa planification stochastique.
2. **Vitesse du Tapis (Belt Speed)** : Lorsque les contraintes temporelles se durcissent (fenêtres de temps plus courtes), SCoBA surpasse largement les méthodes réactives qui n'anticipent pas assez.
3. **Densité de Tâches (Object Probability)** : Face à une charge de travail élevée, SCoBA parvient à mieux répartir les tâches pour éviter la saturation, là où MCTS et Q-Learning peinent à explorer l'espace d'états.

Comme on peut le voir sur ces graphiques (de la Figure 3), les barres orange représentant **SCoBA** se distinguent nettement en restant constamment plus basse que les autres, ce qui indique un taux d'échec plus faible et donc une performance très prometteuse. De plus, l'algorithme démontre une **scalabilité** intéressante : le temps de calcul reste raisonnable pour une exécution en ligne, car la complexité est fonction du nombre de conflits réels et non du nombre total d'agents.

## 2.4 Analyse Critique

**Forces et Garanties** : Contrairement aux approches heuristiques classiques (type glouton ou enchères simples), SCoBA offre des garanties théoriques fortes :

il est prouvé **optimal en espérance** et **complet** (il trouvera une solution si elle existe). Sa structure hiérarchique permet une excellente extensibilité par rapport au nombre de tâches, car la complexité dépend principalement de la **densité des conflits** et non de la taille totale de l'espace d'états.

**Limites et Hypothèses** L'approche repose néanmoins sur des hypothèses trop structurantes :

- **Modèle d'incertitude connu** : L'algorithme suppose que la distribution de probabilité des durées ou des échecs est connue a priori ( $P(\text{succès})$ ), ce qui n'est pas toujours réaliste en environnement inconnu.
- **Sensibilité aux conflits** : Comme pour l'algorithme CBS [17] dont il s'inspire, la performance peut se dégrader dans des environnements très denses (type "*open space*" avec peu de contraintes) où les agents ont une infinité de manières de créer des conflits, augmentant la taille de l'arbre de recherche.
- **Discrétisation** : La construction de l'arbre de politique suppose que l'on peut discrétiser les événements (début/fin de fenêtre), ce qui simplifie la réalité continue.

## 2.5 Conclusion sur l'approche

SCoBA représente l'état de l'art pour les problèmes dominés par des **contraintes temporelles** et une **incertitude** modélisable. En "interdisant" récursivement les conflits plutôt qu'en forçant les affectations, il permet au système de naviguer efficacement dans l'espace des possibles. C'est une illustration parfaite de la manière dont la contrainte (ici, l'exclusion d'une tâche) peut être utilisée comme levier d'optimisation.

Afin de traiter conjointement l'incertitude sur la réalisation des tâches et les contraintes temporelles strictes, l'algorithme **SCoBA** (*Stochastic Conflict-Based Allocation*) [15] s'impose comme une référence dans la littérature récente.

L'analyse des différentes approches d'allocation dynamique de tâches multi-robots met en évidence la complexité et la richesse de ce domaine. La méthode proposée par SCoBA démontre qu'une planification hiérarchique combinant prise en compte de l'**incertitude** et des **contraintes temporelles**, permet d'obtenir des garanties théoriques efficaces, tout en restant **scalable** pour des environnements de taille modérée.

Les approches par contraintes, qu'elles soient liées à la **présence humaine**, ou à la prévention des collisions, montrent l'importance d'intégrer des informations contextuelles directement dans la fonction de coût et/ou la phase d'allocation. Elles améliorent la sécurité, l'efficacité et la prévisibilité, mais restent néanmoins limitées par le nombre d'agents et la structure statique de l'environnement.

Enfin, les approches par incertitude, qu'elles traitent de la **dynamique environnementale** ou des capacités **hétérogènes** des robots, mettent en lumière la nécessité d'une allocation **adaptative** et **flexible**. L'apprentissage et les stratégies *risk-adaptive* permettent de

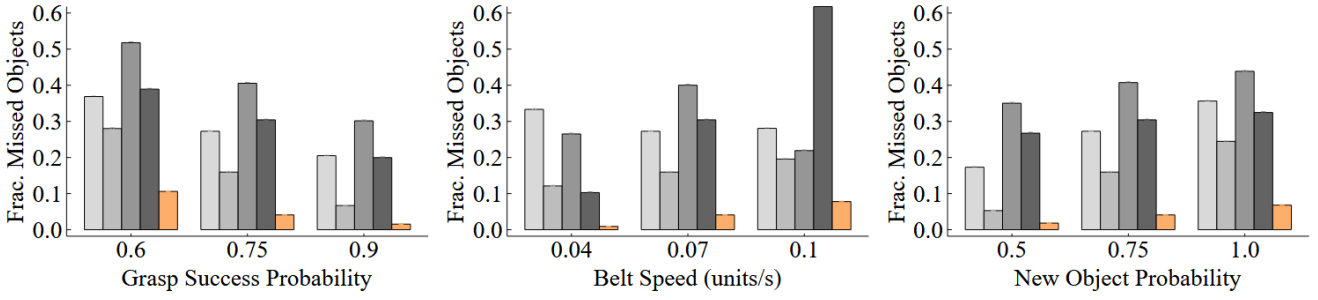


Fig. 5: Legend:  $\square$  EDD  $\square$  Hungarian  $\square$  MCTS  $\square$  Q-Learning  $\square$  SCoBA. On the metric of the fraction of unsuccessful tasks, i.e. objects missed, SCoBA consistently outperforms all other baselines. All results are averaged over 100 trials, with  $T = 500$  time-steps per trial.

FIGURE 3 – [Image des 3 Graphiques de Résultats SCoBA : Grasp Prob, Belt Speed, Object Prob]. Source : [15]

mieux gérer les imprévus, mais reposent sur des hypothèses fortes souvent difficiles à satisfaire dans des environnements réels et très dynamiques.

En synthèse, les travaux récents convergent vers une vision où l'allocation de tâches multi-robots ne se réduit plus à optimiser des critères statiques : il s'agit désormais de gérer simultanément **contraintes**, **incertitudes** et **interactions complexes**, tout en conservant modularité et scalabilité. Parmi les défis encore ouverts, ou non vus ici, figurent l'intégration de modèles d'incertitude plus réalistes, la coordination de grandes équipes de robots ou la cohabitation avec les humains dans des environnements dynamiques, ce qui laisse place à de nouvelles pistes de recherche.

### 3 Approches par contraintes : Contraintes humaines [8]

L'intégration des contraintes humaines dans l'allocation de tâches multi-robots constitue aujourd'hui un enjeu notable pour les systèmes évoluant et opérant dans des environnements partagés avec des agents humains. L'objectif n'est plus seulement d'optimiser la performance, mais de garantir la sécurité et la **prévisibilité** des interactions humain-robot. Cette approche est souvent laissée de côté, mais le framework proposé par *Esikeri, Kyrki, Baumann et Kucner* [8], illustre particulièrement bien cette contrainte.

Les contraintes liées à la présence humaine se distinguent des contraintes classiques (temporelles, énergétiques ou de collision) par leur caractère **dynamique** et leur origine **externe** au système robotique. Contrairement aux robots, les humains ne suivent pas des politiques pragmatiques optimisées pour l'efficacité des tâches, mais adoptent des comportements guidés par des habitudes, des préférences, des règles implicites ou même de part leur libre-arbitre

Cependant, ces comportements ne sont pas purement aléatoires : dans de nombreux environnements réels (espaces publics, boutiques, campus universitaires), les déplacements des humains présentent des **régularités spatio-temporelles** observables, comme des parcours récurrents (à pied ou à bord d'un moyen de locomotion quelconque), ou des zones de fortes affluences à certaines périodes de la journée. Ces régularités offrent un com-

pris réaliste entre imprévisibilité de l'individu et modélisation scientifique et statistique, permettant d'intégrer les contraintes humaines sous la forme de **contraintes dynamiques probabilistes**.

Dans ce cadre, l'approche proposée par *Esikeri et al.* repose sur l'idée que la présence humaine impose des contraintes dynamiques qui doivent être intégrées directement dans la phase d'optimisation. Pour cela, le framework construit, appelé **HATA** pour *Human-Aware Task Allocation*, incorpore des patterns de mouvements humain directement dans le processus d'allocation.

#### 3.1 Modélisation des contraintes humaines

La modélisation des contraintes humaines dans HATA repose sur l'usage de **Maps of Dynamics (MoDs)** : une représentation en grille des schémas probabilistes de déplacement humain dans l'environnement. Ici, on prendra l'exemple de piétons, traversant des chemins partagés par les agents robotiques. Ces MoDs permettent d'estimer, pour chaque zone, la **vraisemblance** qu'un humain soit rencontré dans cette zone au cours de la journée. Ainsi, les préférences humaines ne sont pas définies de manière abstraite, mais dérivent directement de ces motifs de mouvements. L'information extraite des MoDs sert d'input direct à l'algorithme d'allocation.

En effet, dans HATA, ces MoDs agissent directement dans la fonction de coût utilisée pour l'allocation et pénalisent les zones à forte vraisemblance humaine. Cette fonction de coût combine deux composantes essentielles : la longueur du chemin et la **vraisemblance de croiser un humain** sur ce trajet -on ignorera ici la fonction de coût global, qui est une optimisation bayésienne de la longueur des chemins et de cette vraisemblance. Cette vraisemblance peut être écrite comme :

$$P(c_{i,j}, t + \Delta t) = \sum_{k \in K_{i,j}} \frac{T_k}{\Delta t},$$

avec  $K_{i,j}$  l'ensemble des trajectoires piétonnes traversant la cellule  $(i, j)$  pendant l'intervalle  $[t, t + \Delta t)$  et  $T_k$  est la durée que le piéton  $k$  passe dans la cellule  $(i, j)$  au sein de cet intervalle temporel.

## 3.2 Conséquences et résultats

Grâce à cette structure, le système peut **réagir quasi instantanément aux changements humains**. Les auteurs valident leur approche en utilisant un dataset réel de trajectoires piétonnes collectées à Osaka.

Les résultats expérimentaux sont comparés à des méthodes basées sur des plus courts chemins et des allocations à haute densité (HA-Alloc) [19] du papier, montrent :

- une forte diminution des taux d'échecs et des temps de complétion ;
- une diminution des temps d'attente ;
- et une anticipation des zones à haute densité.

Ces résultats confirment que l'ajout maîtrisé de contraintes humaines dans la fonction de coût permet d'améliorer l'efficacité des tâches. Cependant, le framework est **limité à 20 robots**, souffrant de problèmes de coordination dans le framework. Bien que cette intégration des humains constitue déjà une avancée notable, il est probable que des modèles plus complets et précis — intégrant par exemple des modèles individuels et non des densités, ou la collaboration entre humains et robots — puissent encore améliorer l'efficacité du système, et régler le problème de coordination.

## 4 Approches par contraintes : Contraintes de collision [18]

La prise en compte des **collisions** est une contrainte déjà plus étudiée dans la coordination multi-robot, notamment lorsque plusieurs agents opèrent dans un environnement très grand ou structuré. Dans de nombreuses approches classiques que l'on a pu voir au fil de ce papier, les collisions ne sont pas traitées, ou traitées en aval au niveau de la planification des chemins / trajectoires. Cependant, une autre stratégie consiste à intégrer directement cela dans l'allocation dynamique des tâches. L'approche et le framework proposé par *Shit* et *Subudhis* [18] est un parfait exemple de cette philosophie.

### 4.1 Clustering de l'environnement

Le principe fondamental de leur approche repose sur l'idée que les collisions proviennent principalement d'une trop grande **proximité** spatiale ou temporelle entre robots. En découplant les zones de travail, la probabilité que deux agents se trouvent simultanément dans la même région est considérablement réduite. Concrètement, l'environnement est partitionné en plusieurs zones distinctes au moyen d'un **clustering** (*K-means*), chaque cluster regroupant des tâches situées dans une même région. Chaque robot est ensuite assigné à l'un de ces clusters : il devient donc le responsable de toutes les tâches situées dans cette zone géographique.

La probabilité des collisions entre un robot  $i$  et un robot  $j$  est donc formulée ainsi :

$$P(\text{collision}_{i,j}) = \int_T P(r_i \text{ at } x, t) P(r_j \text{ at } x, t) dx dt,$$

où  $T$  désigne l'horizon temporel considéré, et

$P(r_i \text{ at } x, t)$  (resp.  $P(r_j \text{ at } x, t)$ ) la probabilité que le robot  $r_i$  (resp.  $r_j$ ) occupe la position  $x$  au temps  $t$ .

Une fois la zone assignée, chaque robot optimise localement son parcours. Les auteurs utilisent un algorithme de type *2-Opt* pour **minimiser** la longueur de la tournée à l'intérieur du cluster. Cette stratégie est efficace car chaque cluster contient relativement peu de tâches et est généralement d'une taille assez réduite. Les problèmes deviennent alors **locaux** et plus simples à résoudre, tout en garantissant une coordination globale.

Cependant, plusieurs **limites** apparaissent naturellement. Le découpage spatial agit comme une forme de "**distance**" entre les robots plutôt que de complexifier les trajectoires avant d'éviter les collisions. Au lieu de cela, on "interdit" simplement qu'elles puissent se produire, en évitant que deux robots se croisent. On fait aussi l'hypothèse implicite que les robots sont seuls dans leur environnement, ainsi **aucune possibilité de coopération** entre humain-robot n'est étudiée, ce qui est assez paradoxal quand on souligne qu'un des exemple mentionné par *Shit* et *Subudhis* sont les robots de recherche et secours.

De plus, la qualité du partitionnement dépend fortement de la structure spatiale des tâches et de l'initialisation du clustering. Aussi, ce framework suppose que l'on a à exécuter des tâches **homogènes** et **statiques**, ce qui correspond davantage à des environnements industriels (dépôts, usines...) qu'à des scènes dynamiques. En d'autres termes, ce framework est donc particulièrement adaptée aux environnements statiques et/ou faiblement dynamiques, où les zones de travail restent cohérentes dans le temps.

### 4.2 Conséquences et résultats

Les résultats expérimentaux montrent que cette approche **élimine presque totalement les collisions**, tout en maintenant de bonnes performances en termes de temps d'exécution. L'algorithme a été comparé analytiquement à d'autres algorithmes comme le Genetic Algorithm [16] de *Shao* et la méthode gloutonne [4] de *Guo* et *Yu* pour n'en citer que deux. Leur stratégie démontre que la charge de temps computationnelle peut être réduite de 93% par rapport aux autres approches, et la qualité des solutions peut aller jusqu'à être augmentée de 7% par rapport aux méthodes les plus performantes, et bien sûr, il n'y a aucune interaction entre les robots.

Malgré ses limites, cette famille d'approches présente un intérêt important : en traitant la contrainte de collision directement dans l'allocation, elle permet de simplifier considérablement la coordination et d'améliorer la planification. Des méthodes **hybrides** peuvent aussi être pensées, où une première phase de partitionnement global est complétée par des ajustements dynamiques en fonction des événements imprévus - ici, les collisions. Cette direction semble particulièrement prometteuse pour les systèmes multi-robots à grande échelle.

## 5 Approches par incertitudes : Incertitude d'exécution et dynamique de l'environnement

L'un des défis majeurs en allocation dynamique de tâches systèmes multi-robots se trouve dans la gestion de l'incertitude. Elle peut concerner la **dynamique** de l'environnement, la **durée** réelle des tâches, l'**arrivée imprévisible de nouvelles missions**, ou encore l'**évolution** des capacités des robots au cours du temps.

### 5.1 Modélisation de l'incertitude

Dans ce premier article de *Xiaoshan Lin et Roberto Tron* [7], les chercheurs adoptent une vision adaptative où les robots doivent progressivement apprendre la structure véritable de leur environnement. Plutôt que de supposer un modèle parfaitement connu, les auteurs partent du principe que l'incertitude ne peut pas être éliminée et qu'il faut au contraire la **réduire** au fil de l'expérience grâce à de l'**apprentissage par renforcement** [9]. Leur approche repose ainsi sur une architecture bi-niveau. Au niveau supérieur, un module d'allocation cherche un plan satisfaisant des contraintes exprimées en logique temporelle, garantissant que les objectifs à long terme seront respectés. Au niveau inférieur, les robots exécutent le plan tout en apprenant par renforcement le coût réel des actions : temps de déplacement, durées effectives d'exécution, imprécisions liées au terrain ou aux capteurs. L'objectif d'une telle approche est de faire **converger** les robots vers des comportements de plus en plus précis et stables.

Les contraintes temporelles sont exprimées en **Time Window Temporal Logic (TWTL)**, une logique temporelle permettant de spécifier précisément des tâches avec des délais. Chaque formule TWTL est ensuite traduite en un **automate déterministe fini (DFA)** qui accepte exactement les trajectoires qui satisfont la spécification temporelle.

Pour formaliser la dynamique des robots, les auteurs modélisent finalement chaque agent comme un **Labeled Markov Decision Process (LMDP)** défini par :

$$M = (S, A, P, c, AP, L)$$

avec  $S$  l'ensemble des états,  $A$  l'ensemble des actions disponibles,  $P$  la dynamique probabiliste décrivant l'incertitude de transition,  $c$  un coût inconnu initialement, appris en ligne par renforcement,  $AP$  l'ensemble des propositions atomiques et  $L$  une fonction d'étiquetage attribuant à chaque état les propriétés logiques pertinentes.

La composition du DFA avec le LMDP conduit au **product MDP** qui capture simultanément : l'évolution probabiliste du robot et la progression logique vers la satisfaction de la tâche. Sur ce *product MDP*, il devient alors possible de calculer des **distances** vers les états acceptants et de déterminer des **politiques optimales** pour chaque robot, en tenant compte uniquement des informations actuellement disponibles.

### 5.2 Conséquences et résultats

L'approche proposée permet une allocation plus robuste et adaptative, car les politiques apprises s'améliorent au fur et à mesure que les coûts deviennent plus précis grâce à l'apprentissage. Cependant, elle repose sur plusieurs **hypothèses fortes** qui limitent sa portée pratique. D'abord, la modélisation en LMDP suppose que l'espace d'états, les transitions et les étiquetages logiques sont définissables de manière précise, ce qui devient difficile dans des environnements vastes ou continus. Ensuite, la traduction des contraintes en TWTL vers un automate déterministe peut rapidement conduire à une explosion d'états, rendant le **product MDP** difficile à manipuler et limitant l'évolutivité de la méthode. L'article suppose également que chaque robot peut apprendre en ligne des coûts stables et stationnaires, alors que dans des environnements dynamiques, ces coûts peuvent varier ou devenir non-stationnaires. Ces limitations, bien que communes dans la littérature, montrent que l'approche reste surtout adaptée à des environnements structurés et de taille modérée.

## 6 Approches par incertitudes : Incertitude sur les capacités des robots

L'article de *Rudolph, Chernova et Ravichandar* [14] traite de la MRTA dans des équipes de robots **hétérogènes** confrontées à des incertitudes sur leurs capacités. Dans ces contextes, certaines tâches nécessitent la collaboration de plusieurs robots, et le succès global dépend de la coalition choisie. Les approches classiques se montrent insuffisantes : les méthodes *risk-neutral* [13] ignorent le risque et peuvent échouer malgré des prévisions optimistes, tandis que les méthodes *risk-averse* [10] sont trop prudentes et limitent la capacité des robots à constituer des coalitions efficaces.

### 6.1 Modélisation de l'incertitude

Pour surmonter ces limites, les auteurs proposent une stratégie **adaptative** au risque. Cette approche ajuste dynamiquement le comportement des robots entre prudence et prise de risque, selon le contexte et les caractéristiques des tâches à accomplir. L'idée est de **maximiser** la probabilité de succès global en choisissant intelligemment quand être "*conservateur*" et quand tenter des coalitions plus ambitieuses. La modélisation commence par un regroupement de l'équipe robotique en "*espèces*", ou types homogènes. Plutôt que de considérer chaque robot individuellement, les auteurs adoptent une représentation agrégée dans laquelle  $S$  espèces résument la diversité des  $N$  robots. Ce choix permet de réduire la complexité combinatoire tout en conservant la variabilité essentielle entre types de robots - par exemple, UAV et véhicules terrestres. Chaque espèce est caractérisée par un ensemble de traits, autrement dit des capacités telles que la vitesse, la force ou la charge utile. Contrairement à d'autres travaux qui supposent que les robots d'un même type possèdent des traits identiques, les auteurs



modélisent au contraire cette hétérogénéité interne. Ils décrivent chaque tâche comme un ensemble d'exigences (appelées **demandes**) : par exemple un niveau minimal de sécurité, d'efficacité ou de rapidité. Chaque tâche possède aussi un **niveau de risque**. L'idée est que plus la situation est "désespérée", plus les critères de qualité peuvent être relâchés. Le système évalue pour chaque agent et chaque tâche un score combinant utilité (priorité, compétence, gain) et risque (fatigue, danger, incertitude). Il compare ensuite ces scores agent-tâche et choisit l'affectation qui **maximise** la performance tout en **minimisant** le risque global. Ainsi l'agent sélectionné est celui dont le compromis efficacité/sécurité est le plus favorable.

## 6.2 Conséquences et résultats

Les résultats montrent que cette méthode *risk-adaptive* surpasse les approches classiques citées, notamment dans des scénarios simulés de réponse d'urgence. Néanmoins, ce framework repose sur des estimations de risque précises tandis qu'en milieu **réel** ces mesures sont souvent bruitées, partielles ou difficiles à obtenir en continu. Les auteurs supposent aussi que les agents peuvent calculer et communiquer des évaluations fiables de leur état interne, une **hypothèse forte** pour des robots hétérogènes ou déployés en environnements hostiles. Enfin, l'efficacité réelle peut être limitée par les coûts de calcul et la réactivité nécessaire : l'approche fonctionne bien en simulation mais peut être difficile à maintenir en **conditions réelles**, où les risques évoluent très rapidement.

## 7 Conclusion

## Références

- [1] K. A. Athira, J. Divya Udayan, and U. Subramaniam. A systematic literature review on multi-robot task allocation. *ACM Computing Surveys*, 57(3) :68 :1–68 :28, 2025.
- [2] M. Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination : A survey and analysis. *Proceedings of the IEEE*, 94(7) :1257–1270, 2006.
- [3] Brian P. Gerkey and Maja J. Mataric. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9) :939–954, 2004.
- [4] Tian Guo and Jianxiong Yu. Efficient heuristics for multi-robot path planning in crowded environments. arXiv preprint, 2023. arXiv :2306.14409.
- [5] Bilal Kartal, Ernesto Nunes, Julio Godoy, and Maria L. Gini. Monte carlo tree search for multi-robot task allocation. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 4222–4223. AAAI Press, 2016.
- [6] S. Lee, J. Sim, and C. Nam. Very large-scale multi-robot task allocation in challenging environments via robot redistribution. arXiv preprint, 2025. arXiv :2506.07293.
- [7] Xiaoshan Lin and Roberto Tron. Adaptive bi-level multi-robot task allocation and learning under uncertainty with temporal logic constraints. arXiv preprint, 2025. arXiv :2502.10062.
- [8] D. Baumann T.P. Kucner M.K. Eskeri, V. Kyrki. Efficient human-aware task allocation for multi-robot systems in shared environments. arXiv preprint, 2025. arXiv :2508.19731.
- [9] Volodymyr Mnih, Koray Kavukcuoglu, and David Silver. Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533, 2015.
- [10] Cheolhyeon Nam and Dylan A. Shell. Analyzing the sensitivity of the optimal assignment in probabilistic multi-robot task allocation. *IEEE Robotics and Automation Letters*, 2(1) :193–200, 2017.
- [11] Lynne E. Parker. Alliance : An architecture for fault-tolerant cooperative control of heterogeneous mobile robots. *IEEE Transactions on Robotics and Automation*, 14(2) :220–240, 1998.
- [12] Michael Pinedo. *Scheduling*, volume 29. Springer, 2012.
- [13] Amanda Prorok. Redundant robot assignment on graphs with uncertain edge costs. In Nikolaus Correll, Mac Schwager, and Michael Otte, editors, *Distributed Autonomous Robotic Systems*, Springer Proceedings in Advanced Robotics, pages 313–327. Springer, 2019.
- [14] Max Rudolph, Sonia Chernova, and Harish Ravichandar. Desperate times call for desperate measures : Towards risk-adaptive task allocation. arXiv preprint, 2021. arXiv :2108.00346.
- [15] M.J. Kochenderfer D. Sadigh J. Bohg S. Choudhury, J.K. Gupta. Dynamic multi-robot task allocation under uncertainty and temporal constraints. arXiv preprint, 2020. arXiv :2005.13109.
- [16] Juan Shao. Robot path planning method based on genetic algorithm. In *Journal of Physics : Conference Series*, volume 1881, page 022046. IOP Publishing, 2021.
- [17] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent path finding. *Artificial Intelligence*, 219 :40–66, 2015.
- [18] R. C. Shit and S. Subudhi. Multi-robot task allocation for homogeneous tasks with collision avoidance via spatial clustering. arXiv preprint, 2025. arXiv :2505.10073.
- [19] Filip Surma, Tomasz Piotr Kucner, and Masoumeh Mansouri. Multiple robots avoid humans to get the jobs done : An approach to human-aware task allocation. In *2021 10th European Conference on Mobile Robots (ECMR)*. IEEE, 2021.
- [20] Stelios Timotheou. Network flow approaches for an asset task assignment problem with execution uncertainty. In *Distributed Autonomous Robotic Systems 9*, Springer Proceedings in Advanced Robotics, pages 33–38. Springer, 2011.